

To coordinate the sidebar's collapse and expand behavior between an AEM-managed sidebar and a Next.js application, you need an inter-application communication mechanism, as they are likely running as separate entities (possibly in a micro-frontend architecture). 

The most effective methods involve using **browser-native communication channels**: 

1. Using `localStorage` for State Persistence

`localStorage` is a good way to persist the state even after page refreshes and allows different application parts on the same domain to read the data. 

- **AEM Application (Sidebar):** When the user expands or collapses the sidebar, use JavaScript to update a specific `localStorage` item (e.g., `'sidebarState'`).

javascript



```
// In the AEM client library or sidebar script
function toggleSidebar() {
  const isCollapsed = document.body.classList.toggle('collapsed');
  localStorage.setItem('sidebarState', isCollapsed ? 'collapsed' : 'expanded');
  // Dispatch a custom event to notify other apps in real-time
  window.dispatchEvent(new CustomEvent('sidebar-toggled', { detail: { isCollapsed } }));
}
```

- **Next.js Application:** On load, the Next.js app should check `localStorage` for the initial state and subscribe to changes using the `window.addEventListener('storage', ...)` event to react to real-time updates.

javascript



```
// In the Next.js component (e.g., using a useEffect hook)
import { useState, useEffect } from 'react';

const MyPage = () => {
  const [isSidebarCollapsed, setIsSidebarCollapsed] = useState(false);

  useEffect(() => {
    // Initial check on load
    const storedState = localStorage.getItem('sidebarState');
    if (storedState) {
      setIsSidebarCollapsed(storedState === 'collapsed');
    }

    // Listen for storage events across tabs/windows on the same domain
  }, []);
}
```

```
const handleStorageChange = (event) => {
  if (event.key === 'sidebarState') {
    setIsSidebarCollapsed(event.newValue === 'collapsed');
  }
};

window.addEventListener('storage', handleStorageChange);
return () => window.removeEventListener('storage', handleStorageChange);
}, []);

// ... rest of your component
};
```



2. Using Custom Events for Real-time Communication

Custom events are excellent for notifying different parts of the application about changes in real-time without relying on the `storage` event, which only fires across different windows/tabs, not within the same page. Medium +3

- **AEM Application:** Dispatch a `CustomEvent` after changing the sidebar state.

javascript



```
// In the AEM client library
function toggleSidebar() {
  // ... toggle logic
  const isCollapsed = // get current state
  const event = new CustomEvent('aemSidebarToggle', {
    detail: { collapsed: isCollapsed }
  });
  window.dispatchEvent(event);
}
```

- **Next.js Application:** Add an event listener to the `window` object to react to the custom event.

javascript



```
// In the Next.js component
useEffect(() => {
  const handleSidebarToggle = (event) => {
    setIsSidebarCollapsed(event.detail.collapsed);
  }
```

```
};

window.addEventListener('aemSidebarToggle', handleSidebarToggle);
return () => window.removeEventListener('aemSidebarToggle', handleSidebarToggle);
}, []);
```



3. Using a Shared State Library

If both applications are running within the same framework context or shell (e.g., as micro-frontends managed by a shell application), you could potentially use a shared state management library like Redux or Zustand, provided the architecture supports shared dependencies. This is more complex to set up across entirely separate applications but provides a robust, centralized state solution.  DEV Community +2

Summary of Steps:

1. **Define a shared contract:** Agree on the mechanism (`localStorage` key or custom event name and payload structure) to be used for communication.
2. **Implement state updates in AEM:** Add JavaScript logic to the AEM sidebar that dispatches events and/or updates `localStorage` when its state changes.
3. **Implement listeners in Next.js:** Add logic in your Next.js application (likely in a layout component or context provider) to listen for the defined events or changes in `localStorage` and update its own internal state accordingly.
4. **Ensure same domain:** Both applications must be served from the same domain/subdomain for `localStorage` and `window` events to work seamless